

# How to build a Claude-run CRM the "Max" system

---

A GitHub repo as the source of truth, static dashboards that read it, and a Claude agent that runs the rituals. What to build, in what order, and what I'd change after three months live.

Paste this whole thing into a new Claude Project as the opening message. It tells the agent what you're building, in what order, and the mistakes to skip. Section 9 is what I'd change now that I've run it for three months - build v1 as described, but design for v2 from day one.

---

## 1. The idea in one paragraph

A GitHub repo is the single source of truth. A handful of static HTML dashboards (hosted free on GitHub Pages) read JSON data files at runtime and render them - pipeline boards, task board, deals ledger, market panel. A named Claude agent (mine is "Max") sits in a Claude Project with standing instructions and reads/writes those JSON files through the GitHub API. You talk to it - voice notes, screenshots of contact cards, viewing debriefs - and it logs, scores, matches, reminds and keeps a living state file so it never loses the thread between chats. No database, no backend, no SaaS subscription. The software is maybe 30% of it. The other 70% is the rituals in section 5.

---

## 2. The stack

- **Two GitHub repos.** One PUBLIC (needed for free GitHub Pages hosting) holding the dashboards and non-sensitive data. One PRIVATE for anything with personal data at scale - WhatsApp captures, owner lists, chat logs. Rule: never put a phone list or chat log in the public repo. Pages = the open internet.
  - **GitHub Pages** serves the HTML. Deploys lag about 10 minutes - hard-refresh before you debug a "missing" change.
  - **A fine-grained personal access token** scoped ONLY to those two repos (Contents: read/write). The agent holds it in its project instructions and nothing else. Rotate it the moment it appears on a screen, in a screenshot or in a chat. The agent should hold no other credentials - not the server root password, not your bank, nothing.
  - **A Claude Project.** The project instructions are the agent's brain (identity, rules, workflows, file map). A file in the repo, `data/context.md`, is its living memory - current numbers, open threads, decisions, standing rules. The agent reads it at the start of every session and rewrites it whenever something material changes. Chat history is not memory. The repo is.
  - **How it writes:** the agent uses its code/bash tool to call the GitHub Contents API - GET the file (content + sha), edit, PUT back with the sha. Data files first, pages second. It confirms every write in one line: "Done - [what], [where]."
  - **Connectors (add as you go):** Google Calendar (viewings and meetings become debrief prompts), Plaud (meeting recordings to transcripts), Windsor.ai (social stats if you post), PropertyIndex or a DLD open-data pull for market panels, Google Drive.
  - **Optional server:** a \$12/month DigitalOcean droplet running (a) a WhatsApp Web bridge (whatsapp-web.js under pm2) that captures inbound messages to the private repo for the agent to triage, and (b) a cron job pulling Dubai Land Department open-data transactions for your clusters into a JSON the market panel reads. Both are nice-to-have. Build them last.
- 

## 3. The data model - one JSON per thing

**buyers.json** - the active book. One record per person. Fields: - identity: `buyer`, `phone` (THE unique key - dedupe on this), `id`, `source` (portal / Meta ad / referral / IG DM / calling out), `colleague`, `base` (e.g. "UK - weekends only") - want: `area`, `cluster1-3`, `beds`, `budget`, `finance` (Cash / Mortgage / Equity release), `timeline` (Now / 1-3mo / 3-6mo / 6-12mo / Longer), `purpose` (End-user / Investment-yield / Investment-flip / Second home), `segment` (Secondary / Offplan / Distressed) - state: `stage` (New / Qualified / Viewing / Offer / Won / Lost - nothing else), `status`, `score` (1-5, one judgement number set by you - not computed from sub-scores), `matchedUnit`, `lastContact`, `nextTouch` (date), `touchReason`, `nextStep` (MANDATORY - empty means going cold and the dashboard flags it), `order` (1 = top of the board) - notes: `notes` = max 3 high-level bullets; full trail in `history`. `needSummary` one line. - the agent's second opinion: `maxScore` + `maxScoreReason`. When yours and its differ by 1 or more, that shows in the brief. Useful for catching your own bias both ways. - `commPct` per buyer (2% secondary default, 3-4% off-plan) so the money board can rank the book by what it's actually worth.

**archive-buyers.json** - dead and parked. Archiving = move the record, renumber the active orders. Never delete. Every dead buyer gets a one-line loss cause. This file is your revival list.

**prebuyers.json** - the launch bench. Anyone off-plan-curious gets benched here at first mention with budget, timeline, finance and what they'd move for. When ANY new launch appears, the agent cross-checks bench + active + archive and returns ranked candidates with reasons and ready-to-send messages. This is the single most valuable habit in the system and the one I'd build deeper (section 9).

**stock.json** - sellers/listings: unit, cluster, beds, `askPrice`, `bottomLine`, `motivation`, `tenancy`, `access`, seller stage (Longer Term / New / Qualified / Ready / Offer / Won / Lost - "Ready" means you could show it to a buyer today), `source`, `score`, `nextStep`.

**offers.json** - every offer letter: date, buyer, unit, cluster, price, seller, status, notes.

**deals.json** - the closed ledger, newest first (every new deal gets `order` 1, everything else bumps). Per deal: date, unit, buyer, seller, price, both-side commissions, `myBoard` (gross), `myTakeHome` (your split), `payMonth`, `paid`, `paidDate`, sources both sides, segment. Rule: gross deal commission and YOUR rate are different numbers - keep both, never mix them.

**viewings.json** - id, date, buyer, unit, cluster, outcome, `nextStep`. Every viewing logged from the debrief.

**tasks.json** - `{tasks:[], contentIdeas:[]}`. Task: id, title, area, priority, difficulty (easy 1pt / medium 3 / hard 5 / intense 8 - the agent scores it), status, due, created, notes, order, top3 (max three true). The agent, not you, files tasks.

**notes.json** - your notepad by category ("For Jake", "CRM improvements", etc). You ping a line, it files it, you read it before the meeting it's for.

**backlog.json** - every "save that for later". Never on the task board.

**context.md** - the living state file. Numbers, pipeline headline, open threads, standing rules, what's owed. The agent maintains it silently and tells you only when a permanent rule needs pasting into the project instructions.

---

## 4. The pages

- **index.html** - tiles to every page.
- **crm.html** - one real-estate page with tabs:
  - *Today* - the cockpit. Headline strip (month board vs target, active buyers, offers out). Money board: the book ranked by commission value. Warnings: cold buyers (no `nextStep` or no touch in 7 days), overdue touches, score divergence between you and the agent. Score charts that click through to the filtered people list. Due follow-ups today.

- *Pipeline* - kanban by stage, full width. Longer Term column first and excluded from live value. Closed Won strip along the top from the deals ledger.
- *People* - profile per buyer: everything on the record, phone and WhatsApp links, matched stock, market fit (budget vs live stock gap), history trail.
- *Sellers* - kanban by seller stage.
- *Deals* - the ledger with pay months and paid flags.
- *Offers* - offers ledger.
- *Market* - transaction panel fed by the DLD pull (count, median, psf, initial vs resale, biggest, cheapest).
- **tasks.html** - "Mission Control". Top 3 zone, buckets Today / This Week / This Month / Later / Inbox, drag a card to a bucket to set its date, whole-card drag (long-press on mobile), undo, clickable due dates, difficulty-points scoring with today's score.
- Optional: finance (spend vs budget by category), content pipeline, health journal, calls tracker.

**Build rules that cost me real data when broken:** 1. NEVER hardcode data inside a page. Everything reads `data/*.json` at runtime. This rule was broken once and an entire buyer board vanished. 2. Validate JS before pushing ( `node --check` ). Push data files before pages. Bump a `?v=N` cache-buster on the JS file when it changes. 3. Batch find-and-replace edits on HTML fail silently when the pattern doesn't match - apply one edit at a time, verify each, re-pull the file after any failed run. 4. Phone number is the dedupe key. Dupe-guard on create. 5. Pick a brand system on day one (two background greens, a gold accent, a cream text, a serif for headings, a clean sans for body) and never deviate. Consistency is what makes it feel like a product instead of a spreadsheet.

## 5. The operating rituals - this is the actual CRM

Write these into the project instructions. The agent drives them; you don't have to remember.

- **Session start:** read `data/context.md`, then whatever data files the conversation touches.
- **Morning brief (fixed time, first message):** streak, month board vs target, Top 3 with times, due follow-ups (day-3 post-viewing calls, next-touch dates), cold buyers, anything expiring in 24h. Every Today item gets resolved live: do today, re-date, or kill. Nothing sits overdue silently.
- **Evening scoreboard (fixed time):** what moved, calls made, what's about to lapse, tomorrow's Top 3, journal scores if you track them.
- **Intake protocol:** every new lead gets a CALL on arrival. You voice-note the agent after. It logs the person with source, budget, finance, timeline, purpose, base - or parks them on the launch bench. Score on arrival.
- **Viewing debrief:** voice-note style. For each viewer the agent logs the viewing, updates or creates the buyer (score, hard constraints, negotiation plays), creates the day-3 follow-up task, checks duplicates first, flags before merging.
- **Match ritual (both directions):** any new unit mentioned = cross-check the whole book including snoozed and archived, return ranked candidates with reasons and a revival message for anyone forgotten. Any new buyer = check against all live stock. Any new launch = check the bench.
- **Post-viewing day-3 call** for every viewer who didn't offer. **Ghost sweep:** 48h+ silence gets a call, not another text.
- **Callout block** on non-viewing days with a floor (dials or conversations). The agent builds tomorrow's 10-15 name list into every brief with one line why and what to say.
- **Weekly:** Sunday plan (week + content), Friday receipts (what the calls produced). **Monthly:** weakness audit - archived buyers reviewed for false negatives, offers vs closes, call counts.
- **Accuracy rails:** "indicatively / around" until documents confirm. Never invent a seller story. Bank statements beat documents in any money reconciliation.

- **Advisor mandate:** the agent is not a yes-man. It questions numbers that don't add up, challenges weak plans, then executes what you decide.
  - **Confirm every write.** "Done - [what], [where]." If it didn't say Done, it didn't happen.
- 

## 6. How you talk to it

- Voice-note garble is fine. Tell it to parse boldly and ask at most one question when genuinely ambiguous.
  - Screenshot of a WhatsApp contact card + "add to buyers, put in good" = new record, scored, deduped, next touch set.
  - "Callout done, 12 reached, two warm" = logged to the calls tracker, warm ones into the book.
  - "Viewed K-row 4-bed with X, loved the plot, mortgage not approved, wife decides Thursday" = viewing logged, buyer updated, Thursday task created.
  - "Save that for later" = backlog, never the task board.
  - "Remember this - it's a rule" = it tells you to paste it into the project instructions. State changes it handles itself.
- 

## 7. Lessons learned the hard way

- Chat memory is not memory. If it isn't in the repo, it's gone when the chat rolls. Migrating accounts means rebuilding the project from the repo - which is fine, if the repo was maintained.
  - The agent will confidently assume the wrong day or time. Make it verify the clock at session start and before any time-based advice.
  - Tokens on screens, in screenshots, in chats - burned. Rotate.
  - Public repo + personal data = don't. Private repo for PII from day one, not after the first scare.
  - WhatsApp Web automation breaks with every library update. Capture everything, keep the agent as the triage brain, expect to maintain it.
  - Fields you don't capture at intake never get captured. See section 9.
  - Dates go overdue in the second week unless there's a ritual that resolves them daily. Build the checkpoint before you build the board.
  - The agent running ahead and building unasked is as costly as it not building at all. Define the mode: build mode vs coach mode.
- 

## 8. Build order

1. Repo + Pages + one JSON + one page that renders it. Prove the loop.
2. `buyers.json` + the Pipeline kanban. Migrate your current book in by hand - it's the best audit you'll ever do.
3. Claude Project: instructions, `context.md`, the token, the "read context first" rule. First write through the agent.
4. Intake + viewing debrief workflows through chat. Live for a week before adding anything.
5. `tasks.json` + Mission Control.
6. Today cockpit: money board, warnings, due follow-ups.
7. Stock, offers, deals ledger.

8. Calendar + Plaud connectors. Morning brief now includes today's viewings, evening debrief maps to recordings.
9. Match ritual + launch bench. The tagging fields must be populated by now or this step is empty.
10. Market feed (DLD pull) into the Market tab.
11. WhatsApp bridge to the private repo - only if you want inbound triage.

---

## 9. What I'd change now - design v2 in from the start

These are my notes after three months live. Treat them as the spec for your v1, not a later add-on.

1. **Keep the pipeline layout with the tiles.** It works. Don't redesign what's good.
2. **Proper matching is the absolute main thing.** Especially off-plan. Every person tagged at intake with budget, timeline, finance, purpose, base, and what they'd move for - so when any new project or launch comes up, the system produces the send-list in seconds. In my build the fields exist but most records are empty, because capture wasn't enforced at intake. The fix is a capture rule that runs every time, not a cleverer tagging system. This is a whole workstream on its own - scope it as one.
3. **A management system so it's always up to date.** Honest read: beyond capture, the gap was my own management. Design the daily checkpoint (every Today item resolved live: do / re-date / kill) as the first ritual, and make the agent hold the line on it.
4. **Overdue dates.** All mine went overdue. Dates need a re-dating flow, not a rot pile. Nothing should be able to sit overdue silently for more than a day.
5. **Updates in quicker and easier.** Plaud direct from a viewing or meeting recording straight into the profile: recording -> transcript -> agent proposes the update -> one-tap confirm. You talk, the system writes. Same for voice notes.
6. **Today tab.** I never used it well. Rebuild it around the three questions you actually ask at 7:30: who do I call, what's about to lapse, where's the money this month.
7. **Lost and archived need a proper home.** Mine hide as a Closed Lost column plus a collapsible list inside the Offers tab. Give them a People > Lost / Archived view built for revival: last contact, why lost, what would bring them back, one-tap revival message.
8. **Longer term, the CRM moves to its own page or app** as it grows. Not day one, but don't fight the file split when it comes.

---

## 10. Project instructions skeleton for your agent

Copy, rename, fill in:

```
You are [NAME], [my name]'s personal PA, chief of staff and advisor. Direct, no padding. Parse voice-note garble, act on the clear parts, one question max when ambiguous. Never a yes-man: challenge weak plans, question numbers, then execute what I decide.
```

```
SOURCE OF TRUTH: GitHub repo [owner]/[repo], branch main, via the GitHub API with token [fine-grained PAT, these repos only]. Live site: [pages url]. Pages lag ~10 min.
```

```
SESSION START: read data/context.md, then fetch whichever data files the conversation touches. You maintain context.md - update it whenever material facts change. Tell me to update these instructions only when a permanent RULE changes. Confirm every write: "Done - [what], [where]."
```

```
DATA FILES: data/context.md, buyers.json, archive-buyers.json, prebuyers.json (launch / 2 bench), stock.json, offers.json, deals.json, viewings.json, tasks.json, notes.json,
```

backlog.json. [Schemas as in section 3.]

WORKFLOWS: intake protocol · viewing debriefs · match ritual both directions · launch bench check on any new launch · day-3 follow-ups · tasks with area/priority/difficulty/due, Top 3 max three · content ideas to the inbox, never the task board · backlog for "later".

RITUALS: morning brief at [time] (first message) with checkpoint - nothing overdue silently · evening scoreboard at [time] · Sunday plan · Friday receipts · monthly weakness audit. Verify the actual clock before any time-of-day advice.

DASHBOARDS: edit HTML/JS directly. node --check before push. Data files before pages. Bump ?v=N on js changes. NEVER hardcode data in pages - everything reads data/\*.json at runtime.

BRAND: [your palette + fonts].

ACCURACY: "indicatively" until documents confirm. Never invent stories. Bank statements beat documents.

PERSONAL CONTEXT: [what the agent should know about you, your goals, your week.]

Start with sections 1-4 of the build order and live with it for a week before touching anything else. Shout if you get stuck - happy to walk through any of it.

Calum